Увод в програмирането

Оператори за цикъл

ФМИ, специалност „Софтуерно инженерство“

Съдържание

- Оператори за цикъл
  - for
  - while, do-while
- Инструкции за преход
  - break
  - continue
- Вложени цикли

Оператори за цикъл

- Операторите за цикъл се използват за реализиране на изчислителни процеси, при които оператор или група оператори се изпълняват многократно за различни стойности на техни параметри
- Съществуват два вида циклични процеси:
  - Индуктивни
  - Итеративни

Оператори за цикъл … индуктивен (директен) цикличен процес

- Цикличен изчислителен процес, при който броят на повторенията е известен предварително, се нарича индуктивен цикличен процес.
- Пример: По дадени цяло число n и реално число x, да се намери сумата

$$S = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \ldots + \frac{x^n}{n!}.$$

Оператори за цикъл … индуктивен (директен) цикличен процес

- Ако S има начална стойност 1, за да се намери сумата е необходимо n пъти да се повторят следните действия:
  - конструиране на събираемо

$$\frac{x^i}{i!},\ (i = 1, 2, \ldots, n)$$

  - добавяне на събираемото към S.

Оператори за цикъл … итеративен цикличен процес

- Цикличен изчислителен процес, при който броят на повторенията не е известен предварително, се нарича итеративен цикличен процес. При тези циклични процеси, броят на повторенията зависи от някакво условие.
  - Пример: По дадени реални числа x и $\varepsilon > 0$, да се намери сумата

$$s = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots$$

  - където сумирането продължава до добавяне на събираемо, абсолютната стойност на което е по-малка от ε.

Оператори за цикъл … итеративен цикличен процес

- Ако S има начална стойност 1, за да се намери сумата е необходимо да се повторят следните действия:
  - Конструиране на събираемо

$$\frac{x^i}{i!}, (i = 1, 2, \ldots)$$

  - Проверка дали абсолютната стойност на последното добавено към сумата S събираемо е по-малка от ε
  - Ако да, то добавяне на събираемото към S
- В този случай, броят на повторенията зависи от стойностите x и ε

Оператори за цикъл …

- В езика C++ има три оператора за цикъл:
  - *оператори while и do/while*

    Използват се за реализиране на произволни циклични процеси – индуктивни и итеративни
  - *оператор for*

    Чрез него също могат да се реализират произволни циклични процеси, но се използва главно за реализиране на индуктивни циклични процеси.

Оператор за цикъл *while*

Синтаксис

```
while( <условие> )
        <тяло-на-цикъла>
```

Семантика

1. Оценява се <условие>
   A. Ако то е истина се изпълнява <тяло на цикъла> и след това преминаваме обратно на 1.
   B. Ако то не е истина, изпълнението на цикъла се прекратява.

Оператор за цикъл *while*

- *Синтаксис*

  while (<условие>) <оператор>

  където
  - while (докато) е запазена дума;
  - <условие> е булев израз;
  - <оператор> е произволен оператор. Той описва действията, които се повтарят и се нарича **тяло на цикъла**.

Оператор за цикъл *while*

- *Семантика*

  Пресмята се стойността на <условие>. Ако тя е false, изпълнението на оператора while завършва без да се е изпълнило тялото му нито веднъж. В противен случай, изпълнението на <оператор> и пресмятането на стойността на <условие> се повтарят докато <условие> е true. В първия момент, когато <условие> стане false, изпълнението на while завършва

Оператор за цикъл *while*

- *Забележки:*

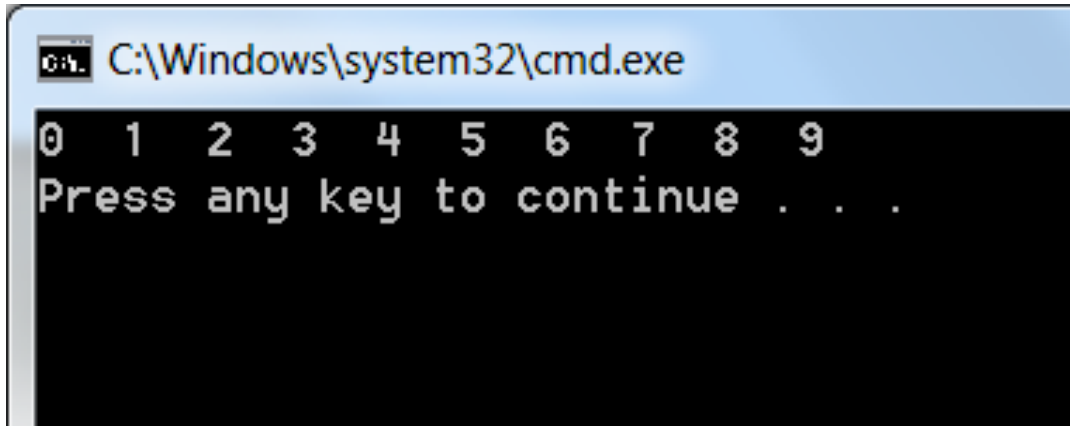
1. Ако е необходимо да се изпълнят многократно няколко оператора, те трябва да се оформят като блок.
2. Следствие разширената интерпретация на true и false, частта <условие> може да бъде и аритметичен израз.
3. Тъй като първото действие, свързано с изпълнението на оператора while, е проверката на условието му, операторът се нарича още **оператор за цикъл с пред-условие.**

Оператор за цикъл *while*

```
int main()
{
    int Counter = 0;

    while (Counter < 10)
    {
        cout << Counter << "   ";
        Counter++;
    }

    cout << endl;
}
```



Оператор за цикъл *do*...*while*

Синтаксис

```
do
   <тяло на цикъла>
while( <условие> );
```

Семантика

1. Изпълнява се <тяло на цикъла>
2. Оценява се <условие>
   A. Ако то е истина преминаваме обратно на 1.
   B. Ако то не е истина, изпълнението на цикъла се прекратява.

Оператор за цикъл *do…while*

- За разлика от **while**, цикълът **do…while**, гарантира поне едно преминаване през тялото на цикъла:

```
int Input = 0;

while(Input > 0)
{
   cout << "*";
   Input--;
}
```

```
int Input = 0;

do
{
   cout << "*";
   Input--;
} while(Input > 0);
```

Оператор за цикъл *do*...*while*

- За разлика от **while**, цикълът **do**…**while**, гарантира поне едно преминаване през тялото на цикъла:

do...while

(намиране на броя цифри на дадено число)

```
int main()
{
    int Value = 0;

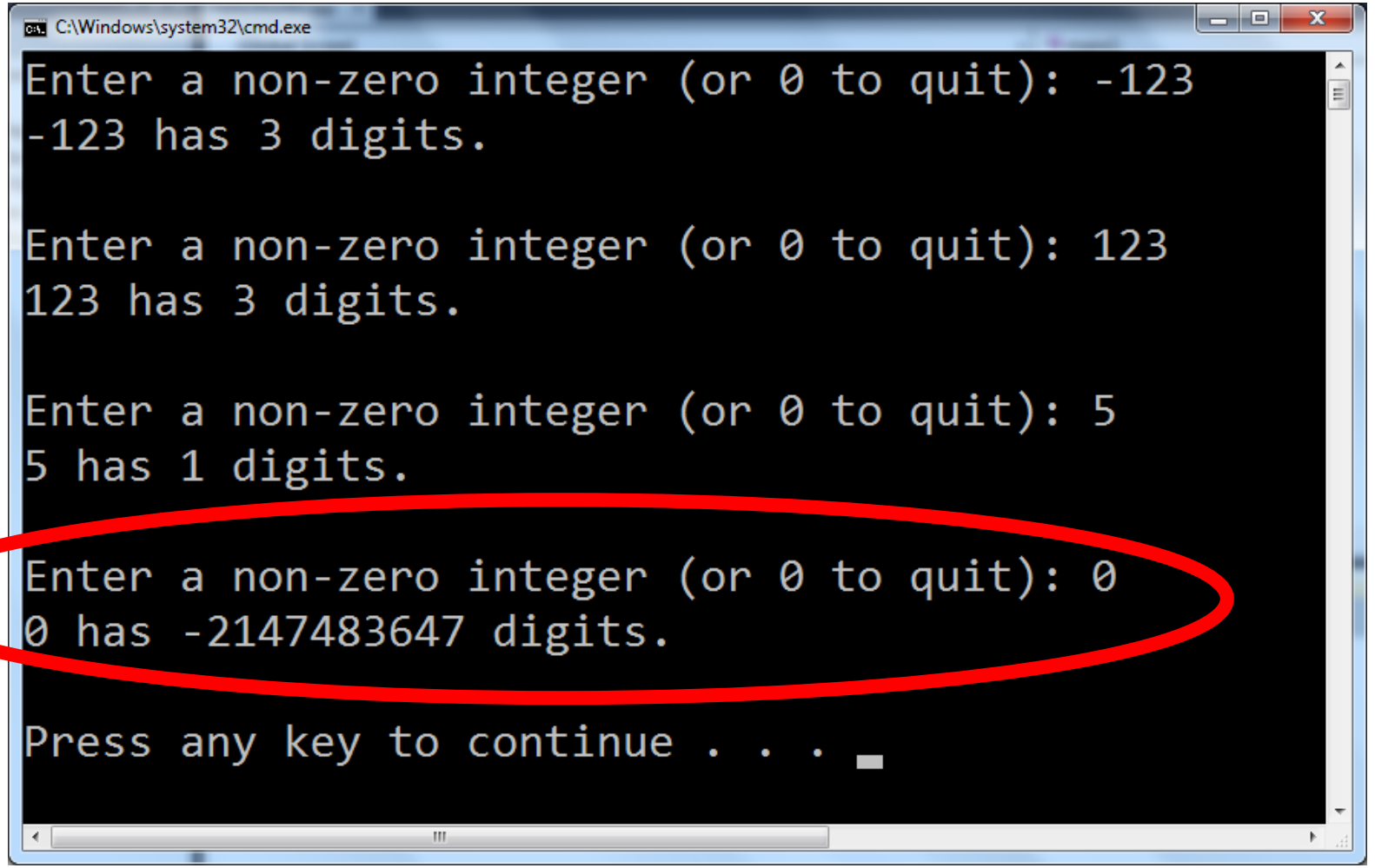
    do {
        cout << "Enter a non-zero integer or 0 to quit: ";
        cin >> Value;

        cout << Value << " has "
         << (int)log10((double)abs(Value)) + 1
         << " digits" << endl << endl;

        } while (Value != 0);

    return 0;
}
```

while & *do*...*while* (пример)



?

while & *do*...*while* (пример)

```
int main()
{
    int Value = 0;

    cout << "Enter a non-zero integer or 0 to quit: ";
    cin >> Value;

    while (Value != 0)
    {
        cout << Value << " has"
         << (int)log10((double)abs(Value)) + 1
         << " digits" << endl << endl;

        cout << "Enter a non-zero integer or 0 to quit: ";
        cin >> Value;
    };
    return 0;
}
```

while & *do*...*while* (пример)

```
int main()
{
    int Value = 0;

    do {
        cout << "Enter a non-zero integer or 0 to quit: ";
        cin >> Value;

        if (Value == 0) break;

        cout << Value << " has"
         << (int)log10((double)abs(Value)) + 1
         << " digits" << endl << endl;

    } while (true);

    return 0;
}
```

Оператор за цикъл *for*

Синтаксис

```
for(<инициализация>; <условие>; <корекция>)
    <тяло на цикъла>
```

Семантика

1. Изпълнява се <инициализация>
2. Оценява се <условие>
   A. Ако то е лъжа, прекратяваме изпълнението на цикъла.
   B. Ако то е истина, преминаваме на 3.
3. Изпълнява се <тяло на цикъла>
4. Изпълнява се <корекция>
5. Преминаваме към 2.

Оператор за цикъл *for*

- *Синтаксис*

  for (<инициализация>; <условие>; <корекция>)

  <оператор>

  където

  - for (за) е запазена дума.
  - <инициализация> е или **точно една** дефиниция с инициализация на една или повече променливи, или един или няколко оператора, **отделени със , и не завършващи с ;.**
  - <условие> е булев израз.
  - <корекция> е един или няколко оператора, **незавършващи с ;**. В случай, че са няколко, отделят се със ,.
  - <оператор> е точно един произволен оператор. Нарича се **тяло на цикъла**.

Оператор за цикъл *for*

- *Семантика*

  Изпълнението започва с изпълнение на частта <инициализация>. След това се намира стойността на <условие>. Ако в резултат се е получило false, изпълнението на оператора for завършва, без тялото да се е изпълнило нито веднъж. В противен случай последователно се повтарят следните действия:
  - Изпълнение на тялото на цикъла;
  - Изпълнение на операторите от частта <корекция>;
  - Пресмятане стойността на <условие>

  докато стойността на <условие> е true.

Оператор за цикъл *for*

- *Забележки:*

1. Тялото на оператора for е точно един оператор. Ако повече оператори трябва да се използват, се оформя блок.
2. Частта <инициализация> се изпълнява само веднъж – в началото на цикъла. Възможно е да се изнесе пред оператора for и остане празна.

```
int factoriel = 1;
int i = 1;
for ( ; i <= n; i++)
    factoriel *= i;          // Намираме n!
```

Оператор за цикъл *for*

```
int factoriel;
int i;
for (factoriel = 1, i = 1; i <= n; )
    factoriel *= i++;
```

- Частта <корекция> се нарича така, тъй като обикновено чрез нея се модифицират стойностите на променливите, инициализирани в частта <инициализация>. Тя може да се премести в тялото на оператора for като се оформи блок от вида {<оператор> <корекция>;

Оператор за цикъл *for*

```
int main()
{
    int Counter = 0;

    while (Counter < 10)
    {
        cout << Counter << "   ";
        Counter++;
    }

    cout << endl;
}
```

```
int main()
{
    for (int Counter = 0; Counter < 10; Counter++)
            cout << Counter << endl;
}
```

Безкраен цикъл *for*

```
// Ако оставим и трите клаузи празни и в тялото
// няма логика, която да го прекрати,
// цикълът ще се изпълнява до безкрай
int Counter = 0;
for (;;)
    cout << Counter << endl;
```

Безкраен цикъл

- Ако в който и да е оператор за цикъл, в <условие> има израз, който никога не приема стойност *false*, то цикълът никога няма да завърши

```
int Counter = 0;
for (;;Counter++)
    cout << Counter << endl;
```

- Добра идея е в тялото на цикъла да се прави нещо, което в някакъв момент ще направи стойността на <условие> равна на *false*

Безкраен цикъл

```
int Counter = 0;

while (Counter < 10)
    cout << Counter << endl;

int Counter = 0;
for (;;)
    cout << Counter << endl;
```

Jump statements (инструкции за преход)

Оператор за цикъл *for*

- Ако частта <условие> е празна, подразбира се true. За да се избегне зацикляне, от тялото на цикъла при определени условия трябва да се излезе принудително, например чрез оператора break.

```
for (int i = 1; ; i++)
    if (i > n)
        break;
    else
        fact = fact * i;
i++; //????
```

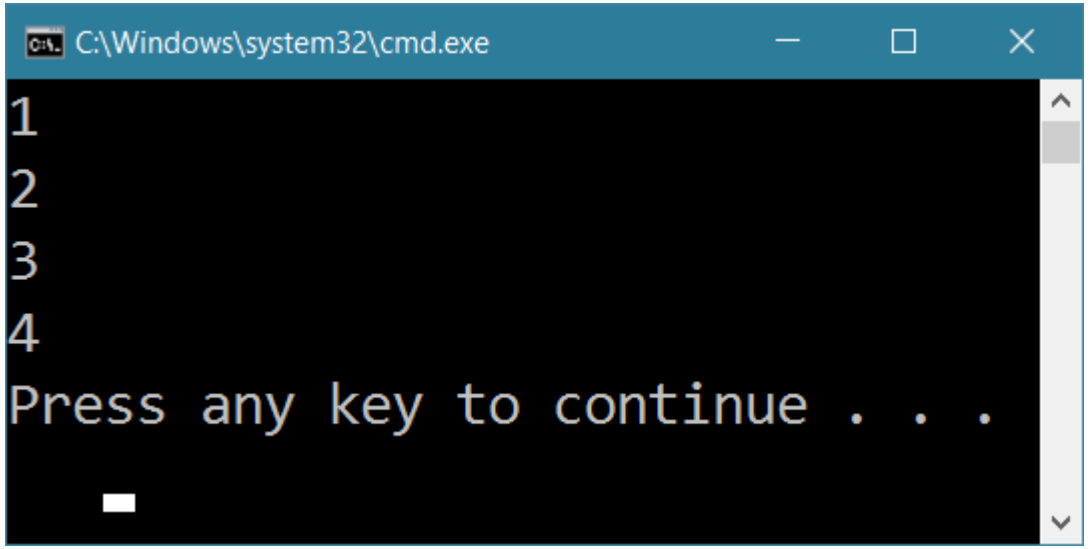
- Област на променливите, дефинирани в заглавната част на for

Оператор *break*

```
int i = 0;

while (++i <= 10)
{
    if (i == 5)
        break;

    cout << i << endl;
}
```

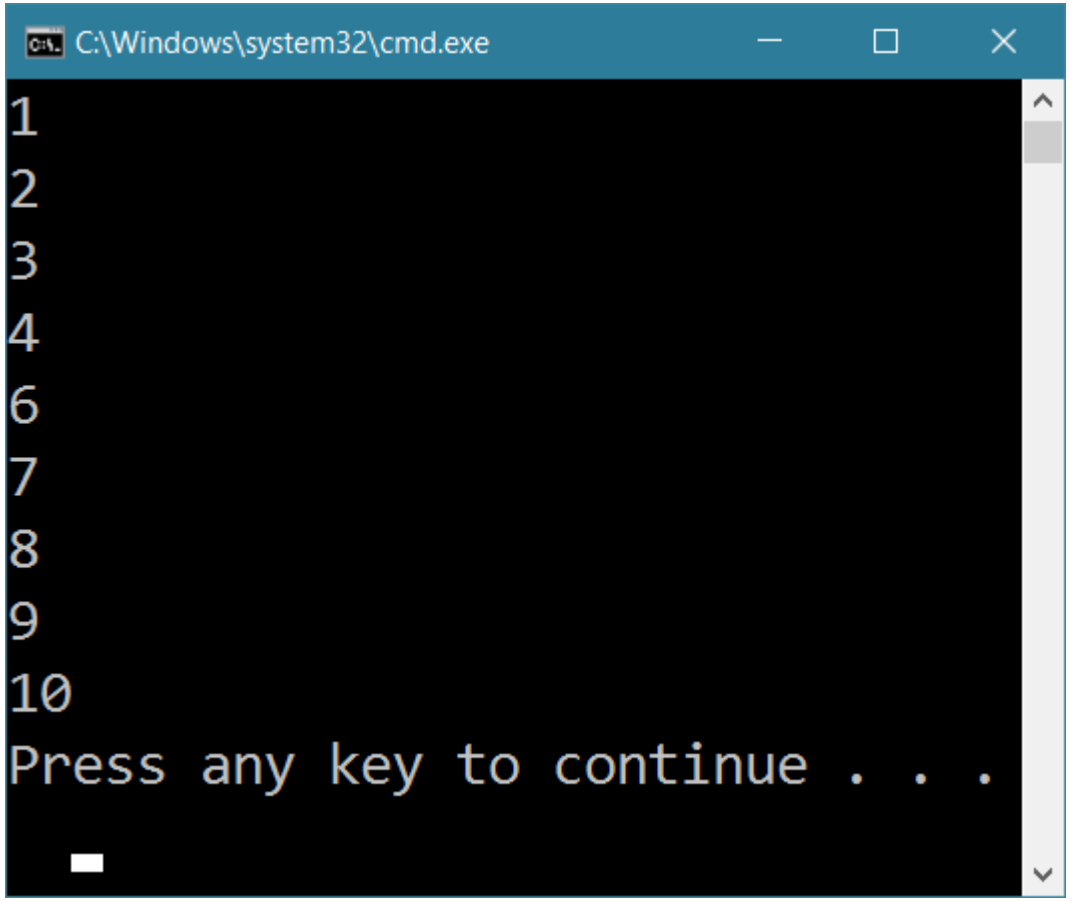


Оператор *continue*

```
int i = 0;

while (++i <= 10)
{
    if (i == 5)
        continue;

    cout << i << endl;
}
```



Оператор *continue*

Ако срещнем continue в **цикъл while**:

1. Оценява се <условие>
2. Ако <условие> е истина, изпълняваме следващата итерация на цикъла.

Ако срещнем continue в **цикъл for**:

1. Изпълнява се <корекция>
2. Оценява се <условие>
3. Ако <условие> е истина, изпълняваме следващата итерация на цикъла.

```
// Когато използвате continue, много внимавайте да не
// предизвиквате безкраен цикъл

int i = 0;

while (i <= 10)
{
    if (i == 5)
        continue;

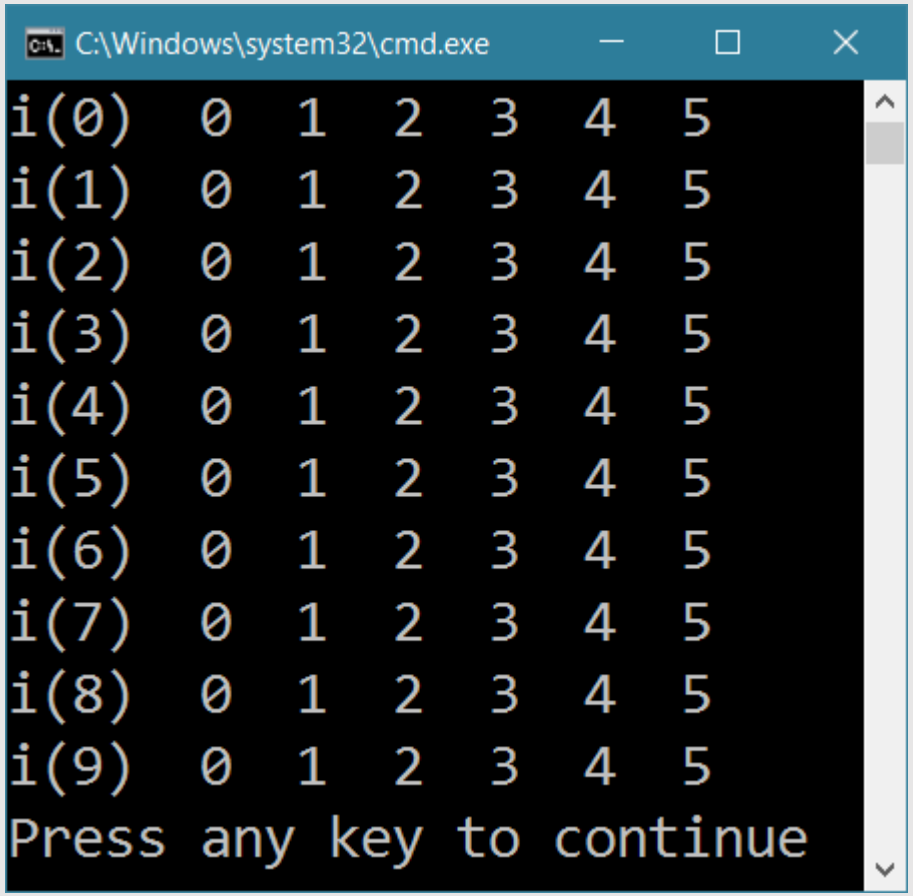
    // някакъв код....

    cout << i++ << endl;
}
```

```
// При вложени цикли, break и continue влияят само
// върху цикъла, в който се намират
for (int i = 0; i < 10; i++)
{
    cout << "i(" << i << ")";

    for (int j = 0; j < 10; j++)
    {
        if (j > 5)
            break;
        cout << setw(3) << j;
    }

    cout << endl;
}
```



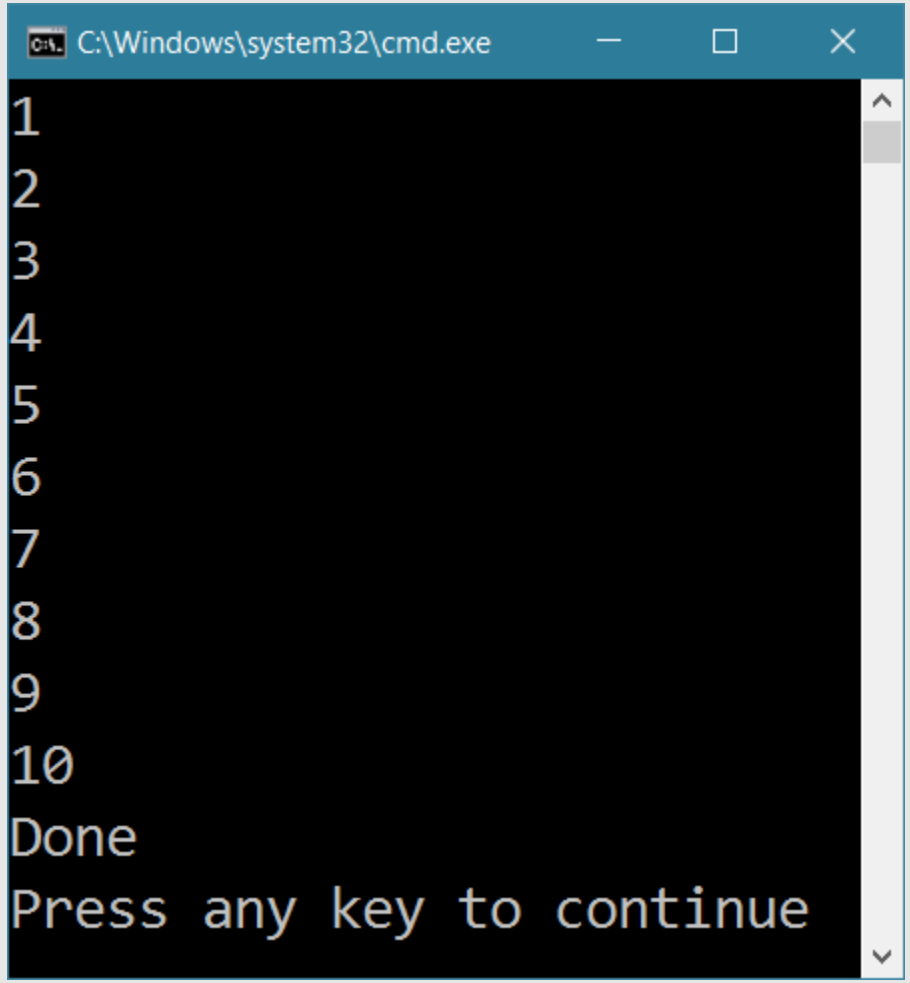
Оператор *goto*

```
int main()
{
    int Counter = 1;

LOOP_START:
    cout << Counter << endl;

    if (Counter < 10)
    {
        Counter++;
        goto LOOP_START;
    }

    cout << "Done\n";
}
```



- Операторът *goto* е отречен още от 60те години на миналия век и практически не се използва
- Освен в много малка част от случаите, когато това наистина се налага

Примери

Пресмятане на факториел

```
int Value = 0;

cout << "Enter an integer: ";
cin >> Value;

long long Factoriel = 1;
int Counter = 1;

while (Counter <= Value)
{
   Factoriel *= Counter;
   Counter++;
}

cout << Value << "! = " << Factoriel << endl;
```

```
int Value = 0;

cout << "Enter an integer: ";
cin >> Value;

long long Factoriel = 1;
int Counter = 1;

while (Counter <= Value)
{
    Factoriel *= Counter++;
}

cout << Value << "! = " << Factoriel << endl;
```

```
int Value = 0;

cout << "Enter an integer: ";
cin >> Value;

int Factoriel = 1;
int Counter = 0;

while (++Counter <= Value)
{
    Factoriel *= Counter;
}

cout << Value << "! = " << Factoriel << endl;
```

```
int Value = 0;

cout << "Enter an integer: ";
cin >> Value;

int Factoriel = 1;
int Counter = 1;

while (Factoriel *= Counter, ++Counter <= Value);


cout << Value << "! = " << Factoriel << endl;
```

```
int Value = 0;

cout << "Enter an integer: ";
cin >> Value;

int Factoriel = 1;

for (int Counter = 1; Counter <= Value; Counter++)
{
    Factoriel *= Counter;
}

cout << Value << "! = " << Factoriel << endl;
```

```
int Value = 0;

cout << "Enter an integer: ";
cin >> Value;

int Factoriel;

for (int Counter = 1, Factoriel = 1;
     Counter <= Value;
     Counter++, Factoriel *= Counter)
;

cout << Value << "! = " << Factoriel << endl;
```

```
int Value = 0;

cout << "Enter an integer: ";
cin >> Value;

int Factoriel = 1;
int Counter = 1;

LOOP:
Factoriel *= Counter;

if (++Counter <= Value)
   goto LOOP;

cout << Value << "! = " << Factoriel << endl;
```

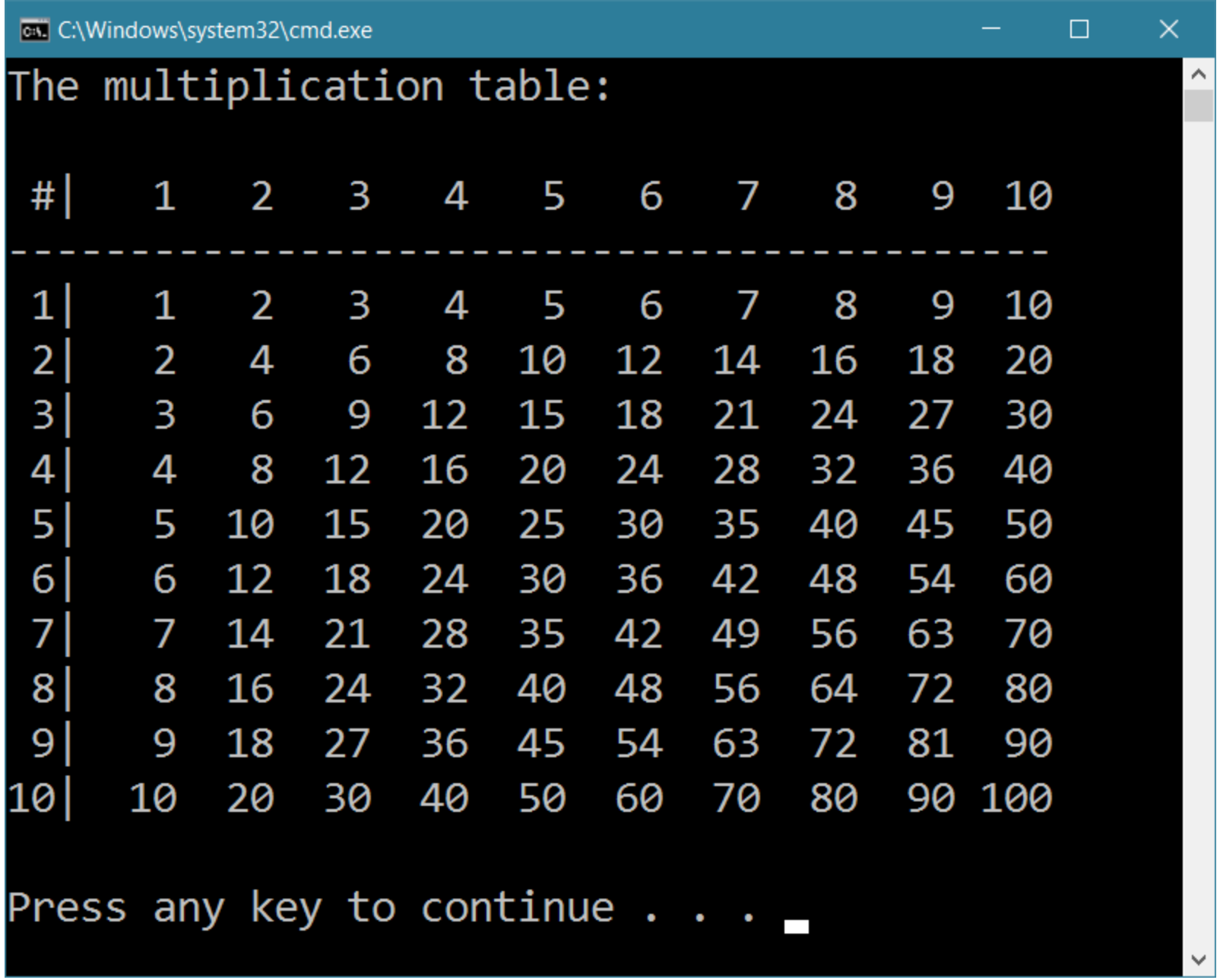
Няколко бележки относно циклите

- Няма нещо, което да можете да направите с for и да не можете да направите с while и обратното.
- Обикновено избираме между двете възможности така, че кодът да бъде оформен по-добре.
- Когато използвате цикли, обикновено има различни начини да напишете един и същ итеративен процес, като пренасяте код между клаузите на цикъла и/или тялото му.
  - В това отношение, цикълът for е най-гъвкав

```
for (int i = 0; i <= 10; i++)
    cout << i << endl;
//-----------------------------------------
int i = 0;
for (; i <= 10; i++)
    cout << i << endl;
//-----------------------------------------
int i = 0;
for (; i <= 10;)
    cout << i++ << endl;
//-----------------------------------------
int i = 0;
for (;;)
{
    if (i > 10) break;
    cout << i++ << endl;
}
//-----------------------------------------
for (int i = 0; i <= 10; cout << i << endl, i++);
```

Вложени цикли

Пример: Генериране на таблицата за умножение


C:\Windows\system32\cmd.exe
The multiplication table:
#| 1 2 3 4 5 6 7 8 9 10
1| 1 2 3 4 5 6 7 8 9 10
2| 2 4 6 8 10 12 14 16 18 20
3| 3 6 9 12 15 18 21 24 27 30
4| 4 8 12 16 20 24 28 32 36 40
5| 5 10 15 20 25 30 35 40 45 50
6| 6 12 18 24 30 36 42 48 54 60
7| 7 14 21 28 35 42 49 56 63 70
8| 8 16 24 32 40 48 56 64 72 80
9| 9 18 27 36 45 54 63 72 81 90
10| 10 20 30 40 50 60 70 80 90 100
Press any key to continue . . .

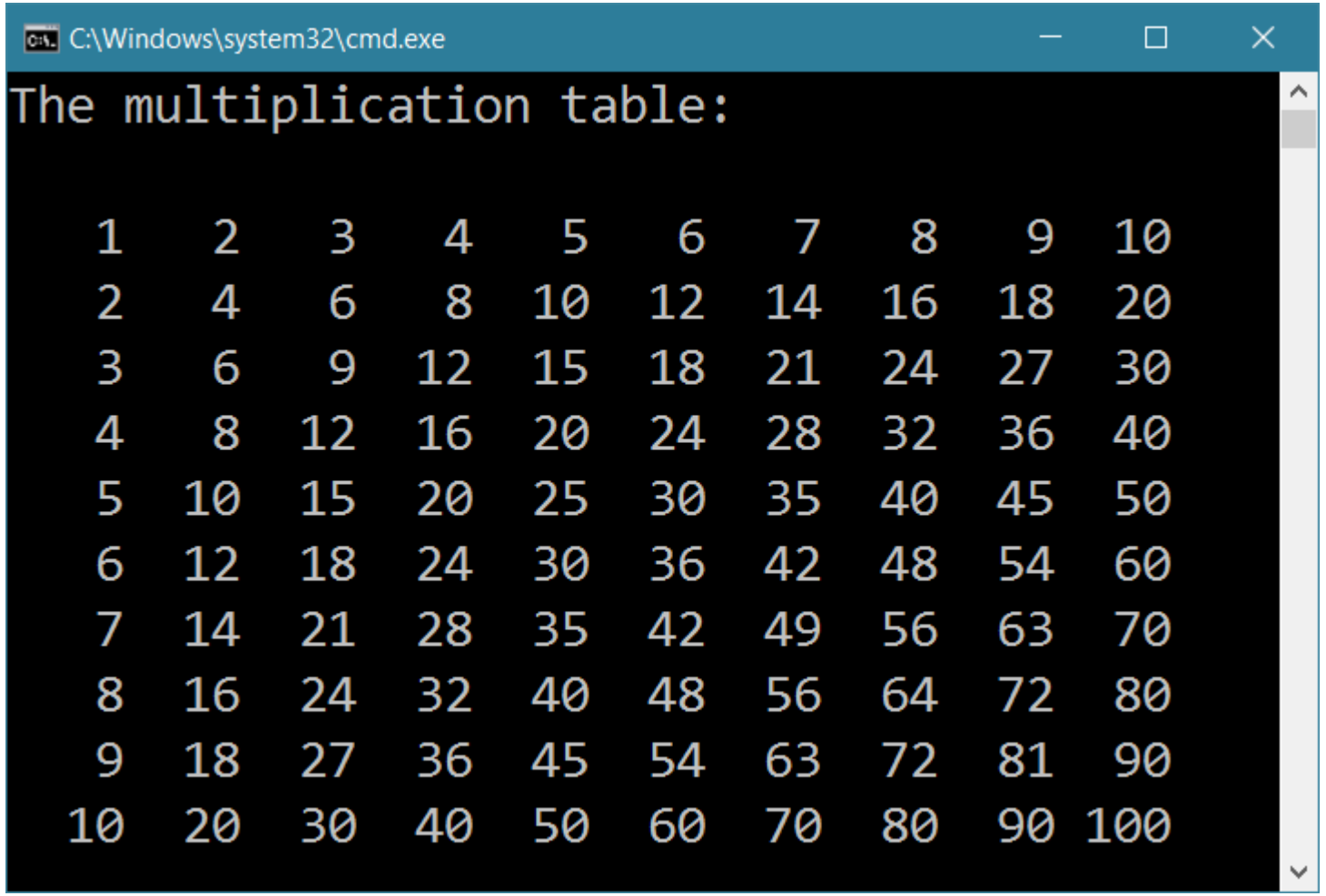
```
cout << "The multiplication table:\n\n";

// Извеждаме таблицата
for (int lhs = 1; lhs <= 10; lhs++)
{
    for (int rhs = 1; rhs <= 10; rhs++)
        cout << setw(4) << (lhs * rhs);

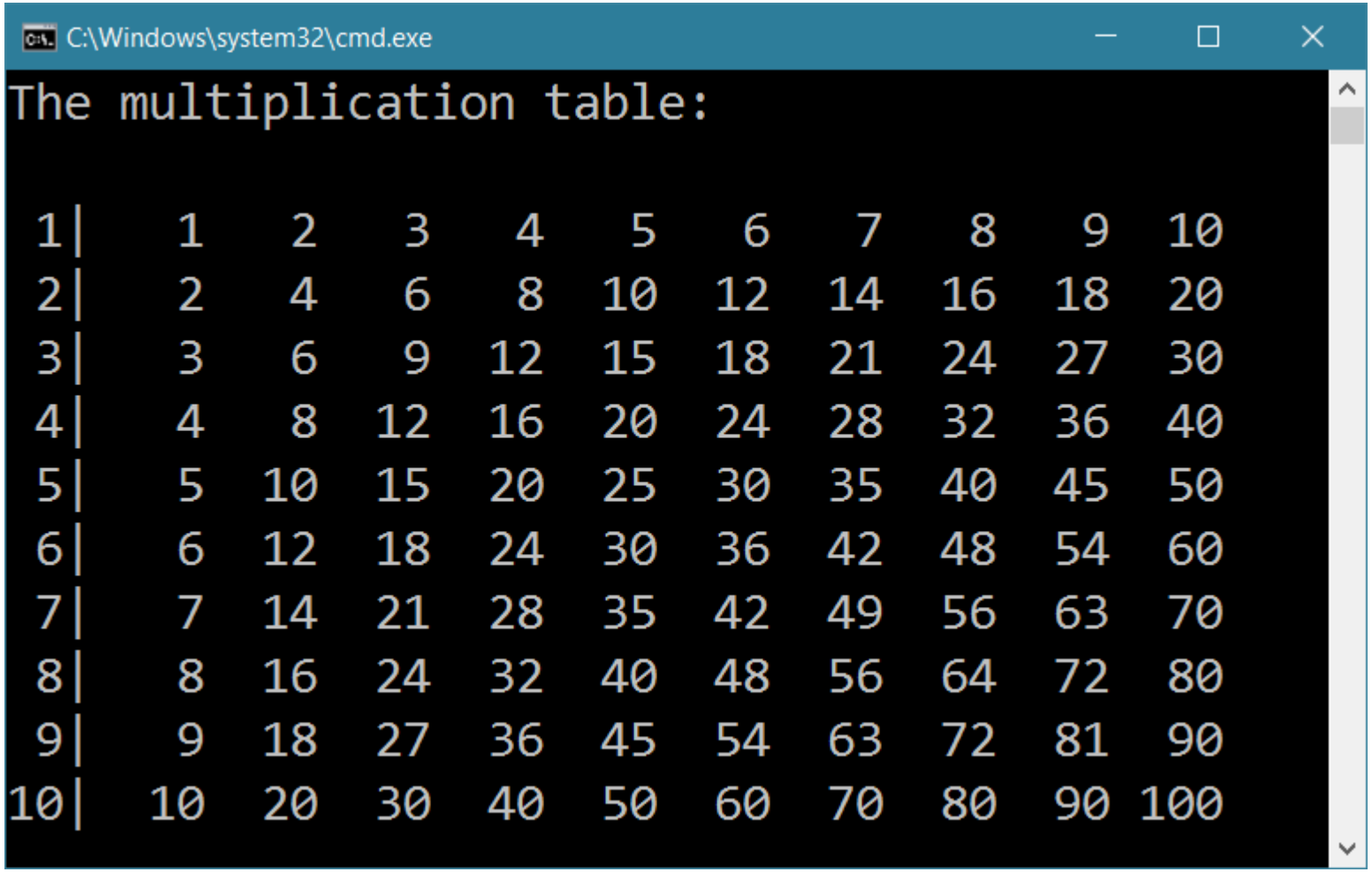
    cout << endl;
}

cout << endl;
```

Стъпка 1: Генериране на таблицата



Стъпка 2: Добавяне на първата колона



```
cout << "The multiplication table:\n\n";

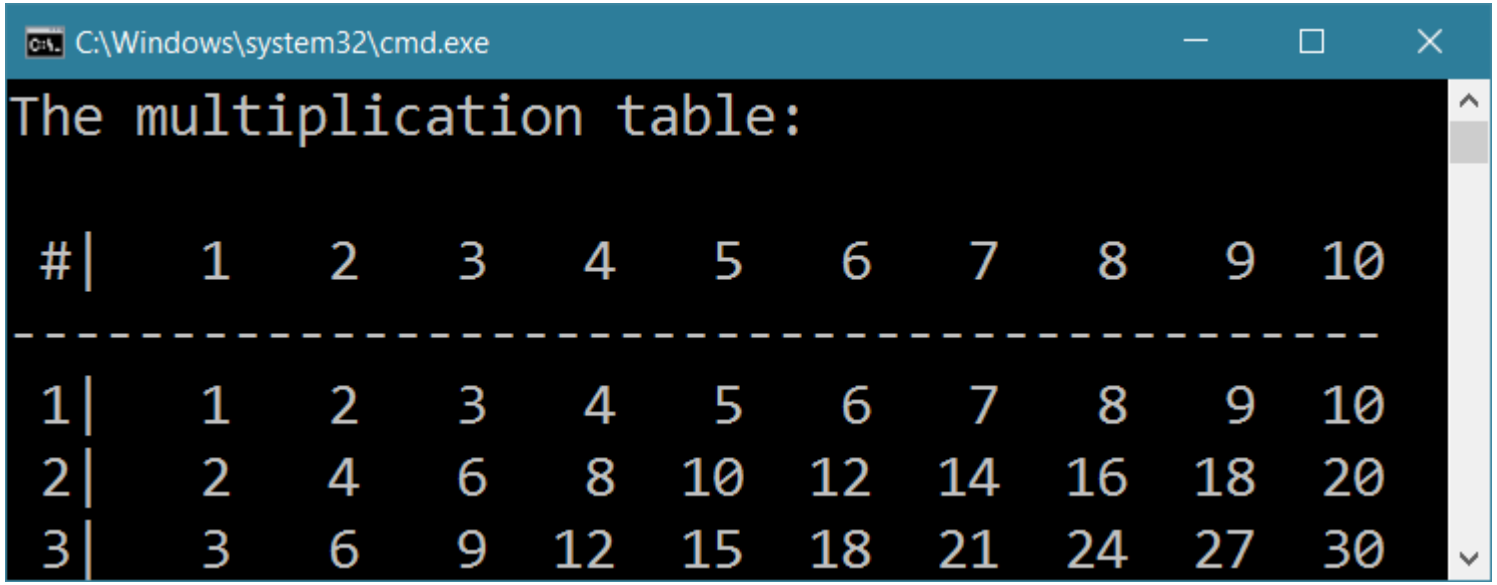
// Извеждаме таблицата
for (int lhs = 1; lhs <= 10; lhs++)
{
    cout << setw(2) << lhs << "|";

    for (int rhs = 1; rhs <= 10; rhs++)
        cout << setw(4) << (lhs * rhs);

    cout << endl;
}

cout << endl;
```

Стъпка 3: Генериране на заглавния ред



```
cout << "The multiplication table:\n\n";

// Извеждаме числата в заглавния ред
cout << " #|";

for (int number = 1; number <= 10; number++)
    cout << setw(4) << number;

cout << endl;

// Извеждаме линията под числата
for (int count = 1; count <= 3 + 10 * 4; count++)
    cout << "-";

cout << endl;
```





```
// Извеждаме таблицата
for (int lhs = 1; lhs <= 10; lhs++)
{
    cout << setw(2) << lhs << '|';

    for (int rhs = 1; rhs <= 10; rhs++)
        cout << setw(4) << (lhs * rhs);

    cout << endl;
}

cout << endl;
```

- В действителност кодът от предишните два слайда не е образец за качество.
- Можете ли да предложите подобрения?
  - Използване на константи!
  - Повтарящ се код??
  - Следващият път ще го подобрим

- За подготовката на тази презентация са използвани слайдове на доц. Атанас Семерджиев